Project 1: Determinant Calculator

9/8/2025

50 Points Possible

Attempt 2

9/8/2025
NEXT UP: Review Feedback

Attempt 2 Score:
N/A

Add Comment

Unlimited Attempts Allowed

Details

Update

Scientific notation and decimal places are fine in the answer.

1.6e+6 will be treated as 1600000
1600000.0 is equivalent.

Introduction

Time to implement an algorithm! In this project, we'll be computing determinants of matrices.

Why? Determinants are useful for many things:

- determining how much a matrix stretches space. This is useful in 3D graphics.
- inverting a matrix, which is important for...
- ...solving a system of equations using Cramer's rule.
- learning whether our matrix will invert the orientation of the coordinate system (left-handed to right-handed and vice versa).

How do we calculate them recursively?

One method is recursive. It's called Laplace expansion

Base case: For a 2-by-2 matrix:
| a b |
| c d | = ad - bc

Another case: For a 3-by-3 matrix:
| a b c |
| d e f | = a*| e f | - b*| d f | + c*| d e |
| g h i | | h i | | g i | | g h |

The smaller matrices are called minors.

Recursive case:
In general, we take each element x of the top row, and multiply it by a matrix made up of all the values ignoring the row and column of x.
After we have calculated all our minors, we compute: det = x0*minor0 - x1*minor1 + x2*minor2 - x3*minor3 + ...

Here is a nice wikipedia article, including a Python code snippet:
https://en.wikipedia.org/wiki/Laplace_expansion

How do we calculate them iteratively?

It turns out, if a matrix is in triangular form (either upper or lower), the determinant can be calculated like this:

| a ? ? ? |
| 0 b ? ? |
| 0 0 c ? | = a * b * c * d
| 0 0 0 d |

That seems way faster! Here's the problem: it only works for triangular matrices.

Can we make a matrix triangular? Yes, we can.

- We can swap one row with another, so we can move zeros to the bottom. Every time we swap, the determinant flips its sign.
- We can scale a row by a scalar. This changes the determinant by the same amount. So if we double a row, the determinant doubles.
- We can add a row, scaled as much as we want, to another row. This, surprisingly, does not change the determinant

The idea is to use these 3 operations to create a triangular matrix. Then, calculate the determinant of the triangular matrix. Then, if you use the first 2 operations, keep track of how you have to change the determinant to make it match the determinant of the original matrix.

This is actually a lot of work. Is it worth it? Only you can find out for sure! Or you can look it up; I can't stop you. However, I strongly suggest trying both methods. If you aren't planning on submitting the Laplace expansion version, it's fine to copy the code from the wikipedia article to see how fast it is and compare with the diagonal matrix technique which you will write yourself. Be careful: do not submit code that you did not write; only submit code that you yourself wrote.

Programming Languages

You must use C. This applies to every project in this class.

Your code will be compiled with -std=c11 -O2 -lm

Input/output

The input will be in standard input. Because of this it does not matter whether we call your program on a redirected file or by manually typing in and hitting the enter button after each line (this is a common question). It will start with a single number describing the dimensionality of the matrix: N.

This number will be followed by N rows, of N space separated floating point numbers (double precision).

You will read this data into your matrix structure (can use a 2-D array, a 1-D array, an array-of-arrays, whatever you want to use).

After computing the determinant of this matrix, you will output it to standard output. Use double precision. Small bits of floating-point inaccuracy are expected and acceptable, because the different approaches to this problem may perform flops in different orders. However, your answer should be within an epsilon of 0.1% of the correct answer.

Don't output anything besides the determinant! You will need to disable debugging print statements to make sure you only output the determinant.

Sample inputs and outputs

1.

```
Input:
2
1 2
0 1

Output:
1
```

2.

```
Input:
3
1 2 3
3 2 1
2 1 3

Output:
-12
```

3.

Input:
6
1 2 3 4 5 1
3 2 1 4 5 1
2 1 3 4 5 1
6 5 4 3 2 1
1 2 1 3 1 4
5 1 5 1 5 1.5551

Output:
-1718.11

Note: these are all samples that will be used to grade your code. See them attached in answer_key.txt.

You may assume that your input is correct. I will not give you malformed or incorrect input. All inputs that you will receive are provided in the attached answer key in the module page.

The output can have decimal places in it. -12.00 will be treated the same as -12.

Each of these samples is a separate file and a separate run of the program. Don't try to read them all in at once. They will all come from standard input redirection. I.e., your program will be run like this:
./your_program < sampleN.txt > outputN.txt

Data Types

Use double-precision floating point variables to store matrix elements.

Scoring

Your base score will be determined by running your code on the 4 answer key matrices, plus all of the samples on this page:

Your score, will be #correct / #total expressed as a percentage. You already have access to all of the samples on this page, as well as the additional matrices posted in the module.

There is a time limit of 5 seconds for each case on the TA's machine/autolab. If your code times out, it counts as a failure. You will certainly want to check to be sure the algorithm you are writing can handle a 30 x 30 matrix to get the full score.

You will receive a small discretionary penalty if you do not read from standard input or if you do not write to standard output. You should not be opening any files in this program.

How to submit

Submit your code as a single C file. If you used multiple C files or a makefile, I'm sorry to make you do this, but please find a way to cram everything into one .c file (Canvas is not really designed for programming projects, and it becomes problematic to work with if we use .zip files).

(no autolab this semester) **Note: you must submit on AutoLab if it is available. We will announce whether you have to submit on AutoLab and where. (https://autolab.encs.vancouver.wsu.edu/courses/CS-450-F24/assessments/Project1)**

Name your main file main.c.

Failure to submit correctly could result in a small penalty (5 to 10 points).

Rules

- Don't output anything besides the determinant. You won't get credit for output that fails the AutoLab tests.
- You may not use matrix processing library functions to solve this problem. You must write the LaPlace expansion and/or triangularization code yourself. If you break this rule, you will not receive credit.

- If your code submission is corrupt, it will be the same as if you did not submit anything at all. You are responsible for checking your submissions afterward by downloading your own submission and running it.
- You must submit your code by following the "how to submit" paragraph. Failure to do this may cause penalties or for your whole submission to not be accepted.

Plagiarism

No plagiarism. It's okay to read about techniques online, but your code must be your own work. You may not work in groups. You may discuss the problem with classmates, but may not share code. Your code may be checked using an automated plagiarism checker. **This is an individual assignment.**

Debugging tips

- Test your determinant against a proper computer algebra system, like Wolfram Alpha. They have an online determinant calculator here: https://www.wolframalpha.com/calculators/determinant-calculator
- Make sure you are parsing input correctly before you investigate your determinant computation. Output everything you read in first, to make sure it's correct. Just make sure to disable this output before submitting.
- You should never get NaN as an answer. If you do, you're probably dividing 0 by 0 when scaling a row. Check that first. Likewise for infinity or negative infinity: it likely means you are dividing a non-zero number by 0.
- I strongly, strongly recommend using valgrind or running your code in a debugger to catch memory errors.

Alignments

- MO2: Code runs without timing out (50 pts)
- MO3: Entire assignment (50 pts)

View Rubric

Project 1 rubric

Criteria	Ratings					Points
MO2 **MO2** view longer description Threshold: 4 pts	**Exceeds Mastery** **5 pts**	**Mastery** **4 pts**	**Near Mastery** **3 pts**	**Below Mastery** **2 pts**	**No Evidence** **1 pts**	--/5 pts
MO3 **MO3** view longer description Threshold: 4 pts	**Exceeds Mastery** **5 pts**	**Mastery** **4 pts**	**Near Mastery** **3 pts**	**Below Mastery** **2 pts**	**No Evidence** **1 pts**	--/5 pts

```c
#include <stdio.h>

int multiplier=1;


void print_matrix(int N, double matrix[N][N])
{
    printf("\n \n");
    for (int i=0; i<N; i++)
    {
        for(int j=0; j<N; j++)
        {
            printf("%0.1lf  ",matrix[i][j]);
        }
        printf("\n");
    }

    printf("\n \n");
}



void Load_matrix(int N, double matrix[N][N])
{

    for (int i=0; i<N; i++)
    {
        for (int j=0; j<N; j++)
        {
            scanf ("%lf", &matrix[i][j]  );
        }

    }


}



void Row_Swap (int N, double matrix[N][N], int R1, int R2)
{   double temp;
    for(int i=0;i<N;i++)
    {
        temp= matrix[R1][i];
        matrix[R1][i]=matrix[R2][i];
        matrix[R2][i]=temp;

    }
    multiplier=multiplier * -1;
}



int row_search(int N, double matrix[N][N], int i  )
{
    for(int j= i+1; j<N; j++)
    {
        if (matrix[j][i] != 0)
        {
```